Sistema di Gestione per un Negozio di Alimentari

Autore: Gino Pandozzi-Trani

Indice

Indice

1 Introduzione al Progetto **3**
- 1.1 Analisi dei Requisiti 4
- 1.2 Descrizione della Traccia 4

2 Progettazione Concettuale **5**
- 2.1 Diagramma ER 5
- 2.2 Diagramma Concettuale 7
- 2.3 Diagramma Concettuale Ristrutturato 8

3 Progettazione Logica **11**
- 3.1 Schema relazionale 11
- 3.2 Dizionario delle entità 12
- 3.3 Associazioni e relazioni 13
- 3.4 Vincoli di integrità 13
- 3.5 Schema logico di funzioni e trigger 15
- 3.6 Schema Logico Dettagliato 17
- 3.7 Trigger e Procedure 17
- 3.8 Flusso di Esecuzione dei Trigger 18

4 Progettazione Fisica **18**
- 4.1 Tabelle SQL 18
- 4.2 Trigger e Funzioni 22
- 4.3 View e Funzioni 27
  - 4.3.1 Funzioni parametrizzate 28
  - 4.3.2 View principali 29
  - 4.3.3 Esempi di utilizzo 30

5 Popolamento e Dati di Esempio **31**

1 Introduzione al Progetto

Sistema di gestione per un negozio di alimentari

Il presente progetto riguarda lo sviluppo di un sistema informativo per la gestione di un negozio di ortofrutta, con l'obiettivo di supportare in modo efficiente le attività quotidiane del punto vendita. Il sistema è pensato per facilitare la gestione delle vendite, del magazzino e dei clienti, fornendo al contempo strumenti per l'analisi delle performance commerciali e l'ottimizzazione dei processi interni.

In particolare, il sistema consente di:

- Gestire i prodotti disponibili, classificandoli per tipologia (frutta, verdura, farinacei, latticini, uova e prodotti confezionati), ciascuna caratterizzata da attributi specifici. Ad esempio, la frutta e la verdura sono associate a una data di raccolta, mentre i latticini riportano la data di mungitura e quella di produzione;
- Registrare i clienti e gestire le relative tessere punti, calcolando automaticamente i punti fedeltà pari al 10% del valore di ogni acquisto;
- Ricercare i clienti in base ai prodotti acquistati e alla quantità di punti accumulati per ciascuna categoria di prodotto;
- Gestire i dipendenti, associando a ciascuna vendita il personale responsabile, con possibilità di analizzare il numero di vendite effettuate e l'introito generato in un determinato periodo.

Il progetto si colloca in un contesto accademico volto a sviluppare competenze nella progettazione e realizzazione di sistemi informativi, con particolare attenzione agli aspetti di modellazione dei dati, efficienza operativa e sicurezza delle informazioni.

1.1 Analisi dei Requisiti

Il sistema deve garantire le seguenti funzionalità principali:

- La gestione dei prodotti, classificati per tipologia (frutta, verdura, farinacei, latticini, uova e prodotti confezionati), ognuna dotata di caratteristiche specifiche;
- La gestione dei clienti, comprendente la registrazione dei dati personali e delle tessere punti;
- La gestione dei dipendenti, con tracciamento delle vendite associate a ciascun membro del personale;
- La generazione di statistiche e report, come l'analisi dei clienti in base alle categorie di prodotti acquistati, ai punti fedeltà accumulati e alle performance dei dipendenti;
- La tutela dei dati sensibili dei clienti e l'affidabilità delle operazioni quotidiane del negozio.

1.2 Descrizione della Traccia

Traccia 5: Sistema di gestione per un negozio di alimentari

Il progetto prevede la realizzazione di un sistema informativo per la gestione di un negozio di ortofrutta. Il sistema deve consentire di gestire le vendite e la disponibilità dei prodotti, identificando le diverse tipologie merceologiche (frutta, verdura, farinacei, latticini, uova e confezionati). Ogni categoria presenta caratteristiche peculiari: ad esempio, la frutta e la verdura includono la data di raccolta, mentre i latticini riportano sia la data di mungitura del latte sia la data di produzione.

Ogni cliente deve essere registrato nel sistema e disporre di una tessera punti che lo identifichi presso il negozio. Per ogni acquisto effettuato, il cliente accumula punti fedeltà pari al 10% del valore della spesa. Il sistema deve inoltre permettere di ricercare i clienti, distinguendoli in base alle categorie di prodotti acquistati e alla quantità di punti maturati per ciascuna tipologia.

Nel caso in cui siano presenti più dipendenti, ogni vendita deve essere associata al dipendente che l'ha conclusa. Deve pertanto essere possibile individuare, in un determinato intervallo temporale, il dipendente che ha effettuato il maggior numero di vendite e quello che ha generato il più alto introito per il negozio.

2 Progettazione Concettuale

2.1 Diagramma ER

Il **diagramma Entità–Relazioni (ER)** descrive la struttura logica delle informazioni gestite dal sistema e le interdipendenze tra le sue principali entità. Il modello è stato progettato per supportare in modo coerente la gestione degli ordini, dei clienti e dei prodotti, garantendo integrità referenziale e tracciabilità delle transazioni.

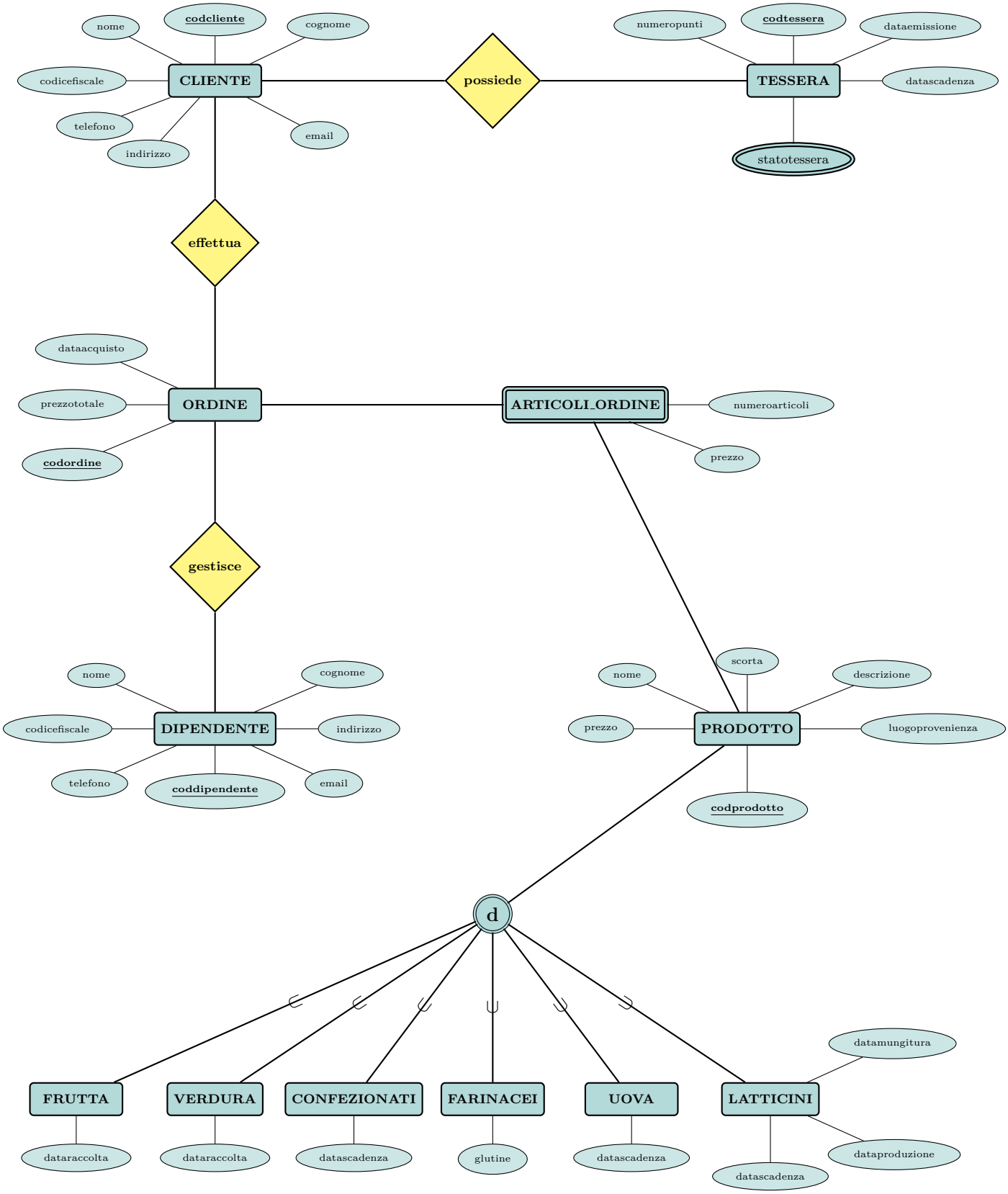
Le **entità principali** individuate sono:

- **CLIENTE**, che rappresenta l'utente che effettua gli acquisti e di cui vengono memorizzati i dati anagrafici e di contatto;
- **TESSERA**, associata univocamente a un cliente, consente di gestire il sistema a punti e il relativo stato operativo (attiva, sospesa, scaduta);
- **ORDINE**, che descrive ciascun acquisto, includendo data, prezzo totale e riferimenti al cliente e al dipendente che lo gestisce;
- **DIPENDENTE**, rappresenta il personale responsabile dell'elaborazione degli ordini;
- **PRODOTTO**, che modella gli articoli in vendita, distinti in categorie merceologiche tramite una specializzazione (*Frutta, Verdura, Latticini*, ecc.);
- **ARTICOLI_ORDINE**, entità debole che concretizza la relazione molti-a-molti tra *Ordine* e *Prodotto*, includendo le informazioni quantitative e di prezzo.

Le **relazioni** principali sono:

- *possiede*, tra **Cliente** e **Tessera** (1:1);
- *effettua*, tra **Cliente** e **Ordine** (1:N);
- *gestisce*, tra **Dipendente** e **Ordine** (1:N);
- *contiene*, tra **Ordine** e **Prodotto** (N:M, tramite **Articoli_Ordine**).

Questo schema ER consente di rappresentare in modo completo i vincoli semantici del dominio, predisponendo la base per la successiva progettazione logica del database relazionale.


nome
codcliente
cognome
codicefiscale
CLIENTE
telefono
indirizzo
email
possiede
numeropunti
codtessera
dataemissione
TESSERA
datascadenza
statotessera
effettua
dataacquisto
prezzototale
ORDINE
codordine
ARTICOLI_ORDINE
numeroarticoli
prezzo
gestisce
nome
cognome
codicefiscale
DIPENDENTE
indirizzo
telefono
coddipendente
email
scorta
nome
descrizione
prezzo
PRODOTTO
luogoprovenienza
codprodotto
d
datamungitura
FRUTTA
VERDURA
CONFEZIONATI
FARINACEI
UOVA
LATTICINI
dataraccolta
dataraccolta
datascadenza
glutine
datascadenza
datascadenza
dataproduzione

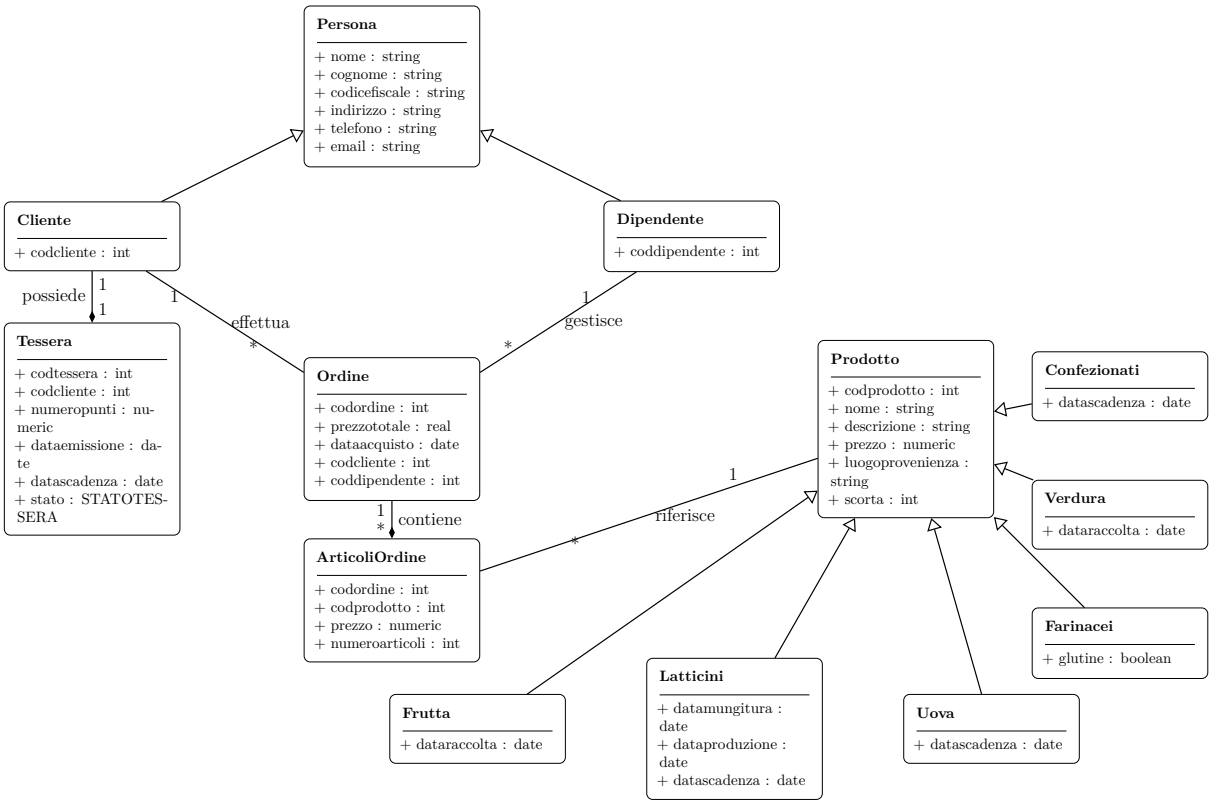
2.2 Diagramma Concettuale

Il **diagramma concettuale** riformula il modello ER in termini di classi e relazioni, secondo una prospettiva orientata agli oggetti. Tale rappresentazione evidenzia la struttura semantica dei dati e le relazioni logiche tra le entità, favorendo la successiva transizione verso il modello logico o implementativo (ad esempio in UML o in codice).

In particolare:

- **Persona** è una *superclasse astratta* che raccoglie gli attributi comuni a **Cliente** e **Dipendente**, riducendo la ridondanza informativa;
- **Cliente** e **Dipendente** derivano da **Persona** ed estendono la classe con un proprio identificativo univoco;
- **Ordine** funge da nodo centrale tra **Cliente** e **Dipendente**, rappresentando la transazione commerciale e mantenendo le associazioni con i prodotti acquistati;
- **ArticoliOrdine** è una *classe di associazione* tra **Ordine** e **Prodotto**, contenente il prezzo unitario e la quantità;
- **Prodotto** è una classe generalizzata che comprende diverse specializzazioni (*Frutta, Verdura, Latticini*, ecc.), ciascuna con attributi specifici coerenti con la categoria merceologica.

Questo modello concettuale consente una visione più astratta e modulare dei dati, facilitando l'estensione del dominio e la successiva mappatura su schemi relazionali o classi software.



2.3 Diagramma Concettuale Ristrutturato

Il **diagramma concettuale ristrutturato** rappresenta una versione ottimizzata e normalizzata del modello concettuale precedente. L'obiettivo della ristrutturazione è ridurre la complessità derivante dalla presenza di numerose specializzazioni, accorpando le categorie di prodotti in un'unica classe più flessibile e parametrizzata.

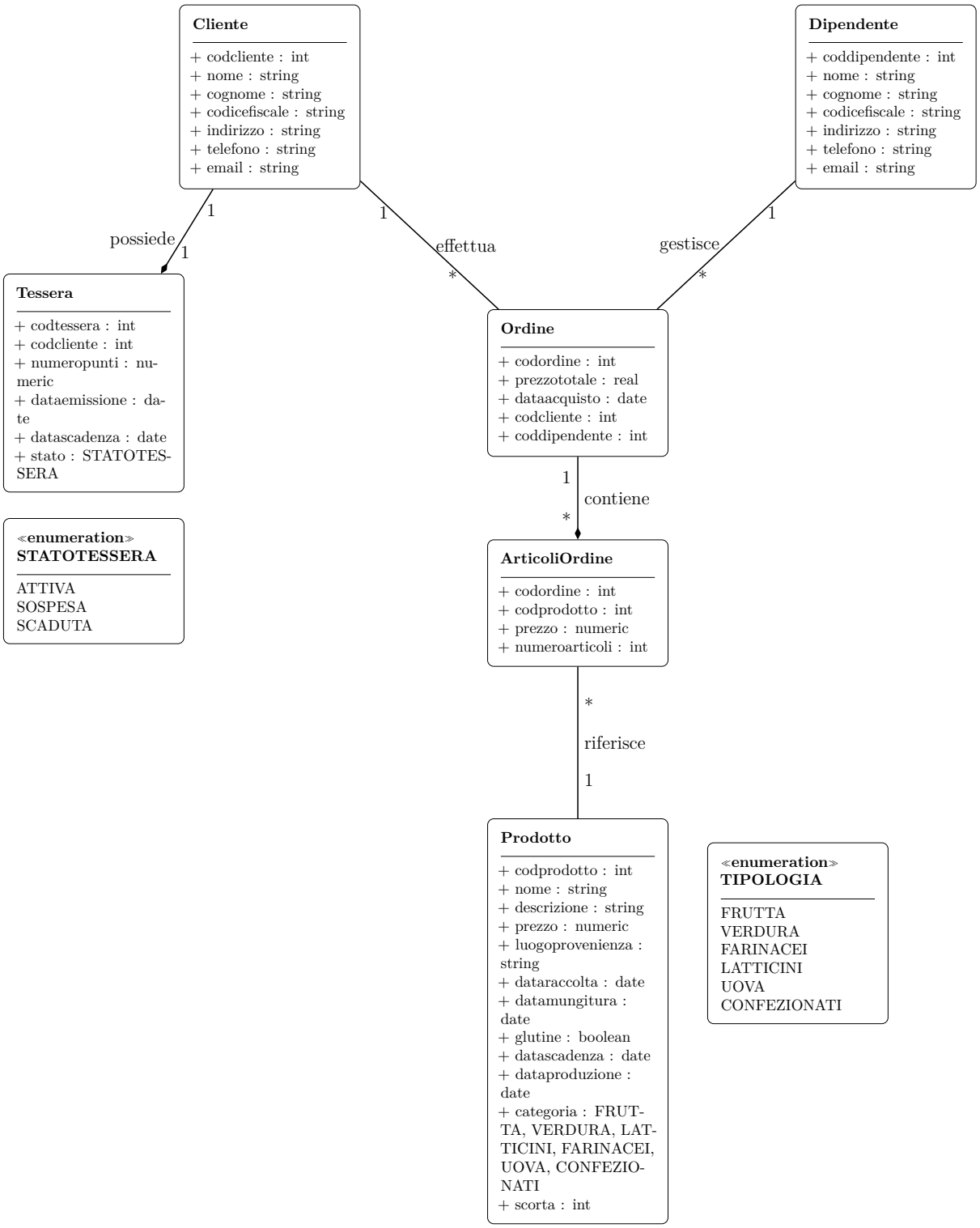
- La **classe `Prodotto`** è stata unificata e arricchita con un attributo enumerativo `categoria`, che consente di distinguere le diverse tipologie di articoli. Gli attributi specifici (ad esempio *data raccolta*, *data mungitura*, *data scadenza*, *presenza glutine*) sono ora integrati nella stessa classe e possono assumere valore nullo quando non pertinenti alla tipologia di prodotto. Questa scelta riduce la complessità del modello e favorisce una maggiore flessibilità nella gestione dei dati;

- Le classi **Cliente** e **Dipendente** sono state accorpate nella **superclasse `Persona`**, dalla quale ereditano gli attributi comuni (*nome*, *cognome*, *codice fiscale*, *indirizzo*, *telefono* ed *email*). Tale ristrutturazione migliora la coerenza semantica e semplifica la gestione delle relazioni con le altre entità, mantenendo la distinzione logica tra clienti e dipendenti;

- Le relazioni tra **Cliente**, **Tessera**, **Ordine** e **ArticoliOrdine** sono state mantenute inalterate, poiché già correttamente rispecchiano i vincoli cardinali e semantici del dominio applicativo, garantendo la tracciabilità e l'integrità delle transazioni;

- Sono state introdotte **enumerazioni** (come `STATOTESSERA` e `TIPOLOGIA`) per rappresentare domini discreti e garantire coerenza, validità e uniformità dei valori nei dati, facilitando al contempo i controlli di integrità a livello applicativo e di database.

Questa versione del modello risulta più compatta e facilmente traducibile in un modello relazionale normalizzato, mantenendo al contempo l'integrità semantica e la chiarezza del design concettuale.


Cliente
+ codcliente : int
+ nome : string
+ cognome : string
+ codicefiscale : string
+ indirizzo : string
+ telefono : string
+ email : string
Dipendente
+ coddipendente : int
+ nome : string
+ cognome : string
+ codicefiscale : string
+ indirizzo : string
+ telefono : string
+ email : string
possiede
1
1
effettua
1
*
gestisce
1
*
Tessera
+ codtessera : int
+ codcliente : int
+ numeropunti : numeric
+ dataemissione : date
+ datascadenza : date
+ stato : STATOTESSERA
Ordine
+ codordine : int
+ prezzototale : real
+ dataacquisto : date
+ codcliente : int
+ coddipendente : int
1
contiene
*
«enumeration»
STATOTESSERA
ATTIVA
SOSPESA
SCADUTA
ArticoliOrdine
+ codordine : int
+ codprodotto : int
+ prezzo : numeric
+ numeroarticoli : int
*
riferisce
1
Prodotto
+ codprodotto : int
+ nome : string
+ descrizione : string
+ prezzo : numeric
+ luogoprovenienza : string
+ dataraccolta : date
+ datamungitura : date
+ glutine : boolean
+ datascadenza : date
+ dataproduzione : date
+ categoria : FRUTTA, VERDURA, LATTICINI, FARINACEI, UOVA, CONFEZIONATI
+ scorta : int
«enumeration»
TIPOLOGIA
FRUTTA
VERDURA
FARINACEI
LATTICINI
UOVA
CONFEZIONATI

3 Progettazione Logica

3.1 Schema relazionale

Lo schema relazionale del database è stato progettato per soddisfare le esigenze di gestione di un negozio di alimentari, garantendo coerenza, integrità dei dati e facilità di interrogazione. Sono stati definiti tipi enumerativi per le categorie di prodotto (`TIPOLOGIA`) e per lo stato della tessera fedeltà (`STATOTESSERA`), assicurando validazione e chiarezza semantica.

Le principali entità (`cliente`, `tessera`, `dipendente`, `prodotto`, `ordine`, `articoliordine`) sono state implementate con vincoli di unicità, integrità referenziale e regole di business. In particolare, la tabella `prodotto` include un vincolo `CHECK` avanzato per garantire che ogni categoria valorizzi solo gli attributi pertinenti, riducendo errori di inserimento.

Note progettuali aggiuntive:

- Le chiavi esterne sono definite con `ON UPDATE CASCADE` per propagare eventuali modifiche dei codici primari.
- Alcune FK, come in `ordine`, hanno `ON DELETE RESTRICT` per evitare cancellazioni involontarie di clienti o dipendenti collegati ad ordini.
- Il tipo `NUMERIC` è utilizzato per prezzi e punti per garantire precisione decimale e correttezza nei calcoli.

Tabella	Attributi principali
cliente	codcliente (PK, SERIAL), nome, cognome, codicefiscale, indirizzo, telefono, email
tessera	codtessera (PK, SERIAL), codcliente (FK, UNIQUE), numeropunti, dataemissione, datascadenza, stato
dipendente	coddipendente (PK, SERIAL), nome, cognome, codicefiscale, indirizzo, telefono, email
prodotto	codprodotto (PK, SERIAL), nome, descrizione, prezzo, luogoprovenienza, dataraccolta, datamungitura, glutine, datascadenza, dataproduzione, categoria, scorta
ordine	codordine (PK, SERIAL), prezzototale, dataacquisto, codcliente (FK), coddipendente (FK)
articoliordine	codordine (PK, FK), codprodotto (PK, FK), prezzo, numeroarticoli

Tabella 1: Schema relazionale: chiavi primarie sottolineate, chiavi esterne doppiamente sottolineate.

3.2 Dizionario delle entità

Ogni tabella rappresenta un elemento chiave del sistema, con attributi principali e vincoli di integrità:

- **cliente**: dati anagrafici dei clienti, validati tramite regex per email e codice fiscale. Chiave primaria `codcliente`.
- **tessera**: tessera punti associata a ciascun cliente (1:1), con saldo punti, stato e date di emissione e scadenza. Chiave primaria `codtessera`, FK su `cliente`.
- **dipendente**: dati anagrafici dei dipendenti, con validazione dei campi. Chiave primaria `coddipendente`.
- **prodotto**: prodotti in vendita, con tipologia, attributi specifici per categoria, prezzo e scorta disponibile. Chiave primaria `codprodotto`. Vincolo `CHECK` avanzato per coerenza categorie.
- **ordine**: registra ogni vendita, collegata a cliente e dipendente. Chiave primaria `codordine`, FK su `cliente` e `dipendente`.
- **articoliordine**: dettaglio dei prodotti acquistati per ogni ordine, con prezzo e quantità. PK composita (`codordine, codprodotto`), FK su `ordine` e `prodotto`.

Entità	Descrizione
cliente	Dati anagrafici dei clienti, con PK `codcliente` e validazioni su codice fiscale e email.
tessera	Tessera punti (1:1) per cliente, con saldo punti, date di emissione/scadenza e stato.
dipendente	Dati anagrafici dei dipendenti, PK `coddipendente`, con validazione dei campi.
prodotto	Prodotti in vendita, con tipologia, caratteristiche specifiche, prezzo e scorta, vincolo `CHECK` avanzato per categorie.
ordine	Rappresenta una vendita, collegata a cliente e dipendente, con calcolo automatico del prezzo totale.
articoliordine	Dettaglio degli articoli per ordine, con prezzo unitario e quantità, PK composita `(codordine, codprodotto)`.

Tabella 2: Entità principali e loro funzione nel database.

3.3 Associazioni e relazioni

Le relazioni tra entità sono implementate tramite chiavi esterne e vincoli di integrità referenziale:

- **Cliente-Tessera** (1:1): ogni cliente possiede una sola tessera. FK con `ON DELETE CASCADE`.
- **Ordine-Cliente** (N:1): ogni ordine è associato a un cliente. FK con `ON DELETE RESTRICT` per protezione dati.
- **Ordine-Dipendente** (N:1): ogni ordine è gestito da un dipendente.
- **Ordine-Prodotto** (N:M): ogni ordine può contenere più prodotti tramite `articoliordine`.

Associazione	Descrizione
Cliente-Tessera	Ogni cliente ha una tessera (1:1), FK con `ON DELETE CASCADE`.
Ordine-Cliente	Ogni ordine è associato a un cliente (N:1), FK con `ON DELETE RESTRICT`.
Ordine-Dipendente	Ogni ordine è gestito da un dipendente (N:1), FK con `ON DELETE RESTRICT`.
Ordine-Prodotto	Ogni ordine contiene più prodotti tramite `articoliordine` (N:M).

Tabella 3: Principali associazioni tra le entità.

3.4 Vincoli di integrità

Il database implementa vincoli completi per garantire la coerenza e la qualità dei dati:

- Vincoli `PRIMARY KEY` e `UNIQUE` per garantire l'unicità dei record.
- Vincoli `FOREIGN KEY` per l'integrità referenziale tra le tabelle.
- Vincoli `CHECK` su valori numerici (`prezzo` $\geq$ `0`, `scorta` $\geq$ `0`), validazione di email e codice fiscale tramite regex.
- Vincolo `CHECK` su `prodotto` per garantire coerenza attributi/categoria.
- Trigger e funzioni associate per:
  - Aggiornamento automatico della scorta e dei punti tessera (`AggiornaScortaEPunti`).

- Aggiornamento del prezzo totale dell'ordine (`AggiornaPrezzoOrdine`, `AggiornaPrezzoOrdineDelete`).
- Verifica della disponibilità della scorta (`VerificaScortaProdotto`).
- Aggiornamento dello stato della tessera (`AggiornaStatoTessera`).
- Ripristino scorta e punti dopo cancellazione articoli ordine (`RipristinaScortaProdotto`).

Vincolo / Trigger	Descrizione
Email Regex	Validazione degli indirizzi email dei clienti e dipendenti.
Codice Fiscale Regex	Validazione secondo il formato italiano standard.
Chiavi Primarie	Unicità dei record in ogni tabella.
Chiavi Esterne	Integrità referenziale tra tabelle correlate.
Vincoli CHECK	Controlli su valori numerici e coerenza attributi per categoria prodotto.
Trigger e Funzioni	Gestione automatica di scorte, punti tessera, prezzo ordine e stato tessera.

Tabella 4: Vincoli di integrità e regole di business implementati nel database.

3.5 Schema logico di funzioni e trigger

La tabella seguente descrive la logica operativa che governa l'interazione tra le **tabelle**, le **funzioni in PL/pgSQL** e i **trigger** del database. Ogni funzione implementa una regola di business automatizzata, mentre i trigger ne determinano l'attivazione a seguito di specifici eventi DML (`INSERT`, `UPDATE`, `DELETE`).

Trigger e Funzioni Implementati

Trigger	Evento	Funzione	Descrizione
`TrgAggiornaScortaPunti`	AFTER INSERT ON `articoliordine`	`AggiornaScortaEPunti()`	Decrementa la scorta del prodotto e incrementa i punti fedeltà del cliente (10% del valore speso). Esempio: se un cliente acquista 3 yogurt a 2€ ciascuno, scorta -3, punti +0,60.
`TrgAggiornaPrezzoOrdine`	AFTER INSERT ON `articoliordine`	`AggiornaPrezzoOrdine()`	Ricalcola il prezzo totale dell'ordine sommando il costo dei nuovi articoli (`prezzo` × `numeroarticoli`).
`TrgVerificaScortaProdotto`	BEFORE INSERT ON `articoliordine`	`VerificaScortaProdotto()`	Verifica che la quantità richiesta non superi la disponibilità. Blocca l'inserimento se insufficiente.
`TrgAggiornaStatoTessera`	BEFORE INSERT OR UPDATE ON `tessera`	`AggiornaStatoTessera()`	Aggiorna lo stato della tessera a "SCADUTA" se la data di scadenza è superata.
`TrgRipristinaScortaPunti`	AFTER DELETE ON `articoliordine`	`RipristinaScortaProdotto()`	Ripristina la scorta e decrementa i punti del cliente dopo la cancellazione di un articolo.
`TrgAggiornaPrezzoOrdineDelete`	AFTER DELETE ON `articoliordine`	`AggiornaPrezzoOrdineDelete()`	Ricalcola il prezzo totale sottraendo il valore degli articoli rimossi (risultato ≥ 0).

Dettaglio delle Funzioni PL/pgSQL

AggiornaScortaEPunti() Questa funzione viene attivata dopo ogni inserimento nella tabella `articoliordine` e svolge due operazioni atomiche:

- Decrementa la scorta del prodotto di `numeroarticoli`
- Aggiunge il 10% del valore della transazione ai punti fedeltà del cliente

AggiornaPrezzoOrdine() Aggiorna il campo `prezzototale` sommando il valore dei nuovi articoli (`prezzo` × `numeroarticoli`).

VerificaScortaProdotto() Funzione di controllo che verifica la disponibilità prima dell'inserimento. Se `scorta < numeroarticoli`, solleva un'eccezione che blocca l'operazione.

AggiornaStatoTessera() Controlla la data di scadenza e, se `datascadenza < CURRENT_DATE`, imposta lo stato a `'SCADUTA'`.

RipristinaScortaProdotto() Funzione compensativa che ripristina la scorta e decrementa i punti fedeltà usando `GREATEST` per evitare valori negativi.

AggiornaPrezzoOrdineDelete() Decrementa il prezzo totale dell'ordine garantendo che il risultato sia sempre ≥ 0 tramite `GREATEST`.

3.6 Schema Logico Dettagliato

3.7 Trigger e Procedure

Panoramica Trigger

- **TrgAggiornaScortaPunti**
  - Evento: AFTER INSERT su articoliordine
  - Funzione: AggiornaScortaEPunti()
  - Scopo: Gestione atomica di scorte e punti fedeltà
  - Esempio: Acquisto di 3 mele a 1.20€:
    - Scorta mele: 50 → 47
    - Punti cliente: +0.36 (10% di 3.60€)
- **TrgAggiornaPrezzoOrdine**
  - Evento: AFTER INSERT su articoliordine
  - Funzione: AggiornaPrezzoOrdine()
  - Scopo: Calcolo automatico del totale ordine
  - Esempio: Aggiunta 3 mele a 1.20€:
    - Totale ordine: +3.60€
- **TrgVerificaScortaProdotto**
  - Evento: BEFORE INSERT su articoliordine
  - Funzione: VerificaScortaProdotto()
  - Scopo: Prevenzione vendite con scorta insufficiente
  - Esempio: Tentativo acquisto 5 mele con scorta 3:
    - Risultato: Errore "Scorta insufficiente"
- **TrgAggiornaStatoTessera**
  - Evento: BEFORE INSERT OR UPDATE su tessera
  - Funzione: AggiornaStatoTessera()
  - Scopo: Gestione automatica scadenza tessere
  - Esempio: Tessera con data scadenza superata:
    - Stato: ATTIVA → SCADUTA

3.8 Flusso di Esecuzione dei Trigger

Sequenza Operazioni

Esempio di inserimento nuovo articolo in un ordine:

1. **TrgVerificaScortaProdotto**
   - Verifica disponibilità prodotto
   - Se insufficiente: blocca operazione
2. **TrgAggiornaScortaPunti**
   - Decrementa scorta prodotto
   - Calcola punti (10% del valore)
   - Aggiorna saldo tessera cliente
3. **TrgAggiornaPrezzoOrdine**
   - Calcola nuovo importo articoli
   - Aggiorna totale ordine

Gestione Errori:

- Rollback automatico in caso di errori
- Ripristino stato precedente
- Messaggi di errore specifici

4 Progettazione Fisica

La progettazione fisica rappresenta la fase in cui il modello logico viene tradotto in strutture concrete all'interno del DBMS scelto. In questa sezione vengono presentati gli script SQL che permettono la creazione delle tabelle, dei vincoli di integrità, delle funzioni, dei trigger e delle viste, garantendo la robustezza, la sicurezza e l'efficienza del sistema informativo. L'obiettivo è fornire una base dati pronta all'uso, ottimizzata per le operazioni di gestione e analisi tipiche di un negozio di alimentari.

4.1 Tabelle SQL

Di seguito sono riportati le tabelle SQL necessarie per la creazione delle principali strutture del database: tipi enumerativi, tabelle, vincoli e relazioni. Ogni script è stato progettato per assicurare la coerenza dei dati e la corretta rappresentazione delle regole di business individuate in fase di analisi.

Enumerazioni

```
CREATE TYPE TIPOLOGIA AS ENUM (
    'FRUTTA', 'VERDURA', 'FARINACEI', 'LATTICINI', 'UOVA', '
       CONFEZIONATI'
);

CREATE TYPE STATOTESSERA AS ENUM (
    'ATTIVA', 'SOSPESA', 'SCADUTA'
);
```

Tabella cliente

```
CREATE TABLE IF NOT EXISTS cliente (
    codcliente SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL CHECK (nome ~* ''^[A-Za-z'' ]+$'),
    cognome VARCHAR(255) NOT NULL CHECK (cognome ~* ''^[A-Za-z''
       ]+$'),
    codicefiscale CHAR(16) NOT NULL UNIQUE CHECK (codicefiscale
       ~* ''^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$'),
    indirizzo VARCHAR(255) NOT NULL,
    telefono VARCHAR(20) CHECK (telefono ~* ''^[0-9+ ]+$'),
    email VARCHAR(255) NOT NULL UNIQUE CHECK (email ~* ''^[A-Za-z0
       -9._%-]+@[A-Za-z0-9.-]+\.[A-Za-z]+$')
);
```

Tabella tessera

```
CREATE TABLE IF NOT EXISTS tessera (
    codtessera SERIAL PRIMARY KEY,
    codcliente INTEGER NOT NULL UNIQUE,
    numeropunti NUMERIC NOT NULL DEFAULT 0.00 CHECK (numeropunti
       >= 0.00),
    dataemissione DATE NOT NULL DEFAULT CURRENT_DATE,
    datascadenza DATE NOT NULL DEFAULT (CURRENT_DATE + INTERVAL '
       2 years'),
    stato STATOTESSERA NOT NULL DEFAULT 'ATTIVA',
    CONSTRAINT tessera_cliente_fk FOREIGN KEY (codcliente)
       REFERENCES cliente (codcliente)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

Tabella dipendente

```
CREATE TABLE IF NOT EXISTS dipendente (
    coddipendente SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL CHECK (nome ~* '^[A-Za-z'' ]+$'),
    cognome VARCHAR(255) NOT NULL CHECK (cognome ~* '^[A-Za-z''
       ]+$'),
    codicefiscale CHAR(16) NOT NULL UNIQUE CHECK (codicefiscale
       ~* '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$'),
    indirizzo VARCHAR(255) NOT NULL,
    telefono VARCHAR(20) CHECK (telefono ~* '^[0-9+ ]+$'),
    email VARCHAR(255) NOT NULL UNIQUE CHECK (email ~* '^[A-Za-z0
       -9._%-]+@[A-Za-z0-9.-]+\.[A-Za-z]+$')
);
```

Tabella prodotto

```
CREATE TABLE IF NOT EXISTS prodotto (
    codprodotto SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL CHECK (nome ~* '^[A-Za-z'' ]+$'),
    descrizione VARCHAR(500),
    prezzo NUMERIC NOT NULL CHECK (prezzo >= 0.00),
    luogoprovenienza VARCHAR(255),
    dataraccolta DATE,
    datamungitura DATE,
    glutine BOOLEAN,
    datascadenza DATE,
    dataproduzione DATE,
    categoria TIPOLOGIA NOT NULL,
    scorta INT NOT NULL DEFAULT 0 CHECK (scorta >= 0)
);
```

Vincolo CHECK su prodotto

```
ALTER TABLE prodotto ADD CONSTRAINT checkCategoria CHECK (
  -- FRUTTA: deve avere data di raccolta
  (categoria = 'FRUTTA' AND dataraccolta IS NOT NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NULL) OR

  -- VERDURA: deve avere data di raccolta
```

```
  (categoria = 'VERDURA' AND dataraccolta IS NOT NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NULL) OR

  -- LATTICINI: devono avere data di mungitura del latte, data di
      produzione e data di scadenza
  (categoria = 'LATTICINI' AND dataraccolta IS NULL AND
     datamungitura IS NOT NULL AND dataproduzione IS NOT NULL AND
     datascadenza IS NOT NULL AND glutine IS NULL) OR

  -- FARINACEI: informazioni sul glutine
  (categoria = 'FARINACEI' AND dataraccolta IS NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NULL AND glutine IS NOT NULL) OR

  -- UOVA: solo data di scadenza
  (categoria = 'UOVA' AND dataraccolta IS NULL AND datamungitura
     IS NULL AND dataproduzione IS NULL AND datascadenza IS NOT
     NULL AND glutine IS NULL) OR

  -- CONFEZIONATI: solo data di scadenza
  (categoria = 'CONFEZIONATI' AND dataraccolta IS NULL AND
     datamungitura IS NULL AND dataproduzione IS NULL AND
     datascadenza IS NOT NULL AND glutine IS NULL)
);
```

Tabella ordine

```
CREATE TABLE IF NOT EXISTS ordine (
    codordine SERIAL PRIMARY KEY,
    prezzototale NUMERIC NOT NULL DEFAULT 0.00 CHECK (
       prezzototale >= 0),
    dataacquisto DATE NOT NULL,
    codcliente INTEGER NOT NULL,
    coddipendente INTEGER NOT NULL,
    CONSTRAINT ordineclientefk FOREIGN KEY (codcliente)
       REFERENCES cliente (codcliente)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT ordinedipendentefk FOREIGN KEY (coddipendente)
       REFERENCES dipendente (coddipendente)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);
```

Tabella articoliordine

```
CREATE TABLE IF NOT EXISTS articoliordine (
    codordine INTEGER NOT NULL,
    codprodotto INTEGER NOT NULL,
    prezzo NUMERIC NOT NULL CHECK (prezzo >= 0.00),
    numeroarticoli INT NOT NULL CHECK (numeroarticoli > 0),
    CONSTRAINT PK_ArticoliOrdine PRIMARY KEY (codordine,
       codprodotto),
    CONSTRAINT articoliordineprodottofk FOREIGN KEY (codprodotto)
        REFERENCES prodotto (codprodotto)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT articoliordineordinek FOREIGN KEY (codordine)
       REFERENCES ordine (codordine)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);
```

4.2 Trigger e Funzioni

In questa sezione vengono presentati i principali trigger e funzioni che automatizzano la gestione dei dati e garantiscono il rispetto delle regole di integrità. I trigger agiscono automaticamente su eventi specifici (inserimenti, aggiornamenti, cancellazioni), mentre le funzioni eseguono calcoli e controlli complessi, migliorando la qualità e la sicurezza delle operazioni.

Nota Importante

L'utilizzo di trigger e funzioni consente di:

- Aggiornare automaticamente la scorta dei prodotti e i punti fedeltà dei clienti.
- Verificare la disponibilità dei prodotti prima di ogni vendita.
- Mantenere la coerenza tra ordini, articoli e tessere punti.
- Prevenire errori e garantire la robustezza del sistema.

Panoramica dei Trigger

Di seguito una panoramica dei principali trigger implementati, con una breve descrizione del loro scopo:

- **TrgAggiornaScortaPunti**: aggiorna la scorta del prodotto e i punti della tessera cliente dopo l'inserimento di un articolo in un ordine.
- **TrgAggiornaPrezzoOrdine**: aggiorna il prezzo totale dell'ordine dopo l'inserimento di un articolo.
- **TrgVerificaScortaProdotto**: verifica che la scorta sia sufficiente prima di inserire un articolo nell'ordine.
- **TrgAggiornaStatoTessera**: aggiorna lo stato della tessera (es. scaduta) in base alla data di scadenza.
- **TrgRipristinaScortaPunti**: ripristina la scorta e aggiorna i punti tessera dopo la cancellazione di un articolo da un ordine.
- **TrgAggiornaPrezzoOrdineDelete**: aggiorna il prezzo totale dell'ordine dopo la cancellazione di un articolo.

Esempio pratico:

Quando un cliente acquista 3 mele (prodotto con scorta 10), il trigger `TrgAggiornaScortaPunti` riduce la scorta a 7 e aggiunge il 10% del valore speso ai punti della tessera associata. Se la scorta fosse insufficiente, il trigger `TrgVerificaScortaProdotto` bloccherebbe l'operazione.

Trigger principali

```
-- Trigger per aggiornare scorta e punti tessera dopo inserimento
    articolo
CREATE TRIGGER TrgAggiornaScortaPunti
AFTER INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaScortaEPunti();

-- Trigger per aggiornare il prezzo totale dell'ordine dopo
   inserimento articolo
CREATE TRIGGER TrgAggiornaPrezzoOrdine
AFTER INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaPrezzoOrdine();

-- Trigger per verificare la scorta prima dell'inserimento
CREATE TRIGGER TrgVerificaScortaProdotto
BEFORE INSERT ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION VerificaScortaProdotto();
```

```sql
-- Trigger per aggiornare lo stato della tessera
CREATE TRIGGER TrgAggiornaStatoTessera
BEFORE INSERT OR UPDATE ON tessera
FOR EACH ROW
EXECUTE FUNCTION AggiornaStatoTessera();

-- Trigger per ripristinare scorta e punti tessera dopo DELETE da
    articoliordine
CREATE TRIGGER TrgRipristinaScortaPunti
AFTER DELETE ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION RipristinaScortaProdotto();

-- Trigger per aggiornare il prezzo totale dell'ordine dopo
   DELETE da articoliordine
CREATE TRIGGER TrgAggiornaPrezzoOrdineDelete
AFTER DELETE ON articoliordine
FOR EACH ROW
EXECUTE FUNCTION AggiornaPrezzoOrdineDelete();
```

Descrizione delle Funzioni

Ogni funzione associata ai trigger svolge un compito specifico. Di seguito una breve spiegazione prima del codice:

- **AggiornaScortaEPunti**: aggiorna la scorta del prodotto e incrementa i punti della tessera cliente in base al valore speso.
- **AggiornaPrezzoOrdine**: aggiorna il totale dell'ordine dopo l'inserimento di un articolo.
- **VerificaScortaProdotto**: controlla che la scorta sia sufficiente prima di confermare la vendita.
- **AggiornaStatoTessera**: imposta lo stato della tessera a "SCADUTA" se la data di scadenza è superata.
- **RipristinaScortaProdotto**: ripristina la scorta e aggiorna i punti tessera dopo la cancellazione di un articolo.
- **AggiornaPrezzoOrdineDelete**: aggiorna il totale dell'ordine dopo la cancellazione di un articolo.

Per ogni funzione, il codice è preceduto da una breve descrizione e, dove utile, da un esempio pratico di utilizzo.

Funzione AggiornaScortaEPunti

```
CREATE OR REPLACE FUNCTION AggiornaScortaEPunti()
RETURNS TRIGGER AS $$
DECLARE
    punti_da_aggiungere NUMERIC;
    cod_cliente INTEGER;
BEGIN
    -- Aggiorna la scorta del prodotto
    UPDATE prodotto
    SET scorta = scorta - NEW.numeroarticoli
    WHERE codprodotto = NEW.codprodotto;

    -- Calcola i punti da aggiungere (10% del valore speso)
    punti_da_aggiungere := ROUND(NEW.prezzo * NEW.numeroarticoli
       * 0.10, 2);

    -- Recupera il cliente associato all'ordine
    SELECT codcliente INTO cod_cliente
    FROM ordine
    WHERE codordine = NEW.codordine;

    -- Aggiorna i punti della tessera del cliente
    UPDATE tessera
    SET numeropunti = numeropunti + punti_da_aggiungere
    WHERE codcliente = cod_cliente;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaPrezzoOrdine

```
CREATE OR REPLACE FUNCTION AggiornaPrezzoOrdine()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ordine
    SET prezzototale = prezzototale + (NEW.prezzo * NEW.
       numeroarticoli)
    WHERE codordine = NEW.codordine;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione VerificaScortaProdotto

```
CREATE OR REPLACE FUNCTION VerificaScortaProdotto()
RETURNS TRIGGER AS $$
DECLARE
    scorta_corrente INT;
BEGIN
    SELECT scorta INTO scorta_corrente FROM prodotto WHERE
       codprodotto = NEW.codprodotto;
    IF scorta_corrente < NEW.numeroarticoli THEN
        RAISE EXCEPTION 'Scorta insufficiente per il prodotto %:
           richiesta % - disponibile %', NEW.codprodotto, NEW.
           numeroarticoli, scorta_corrente;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaStatoTessera

```
CREATE OR REPLACE FUNCTION AggiornaStatoTessera()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.datascadenza < CURRENT_DATE THEN
        NEW.stato := 'SCADUTA';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Funzione RipristinaScortaProdotto

```
CREATE OR REPLACE FUNCTION RipristinaScortaProdotto()
RETURNS TRIGGER AS $$
DECLARE
    cod_cliente INTEGER;
    punti_da_togliere NUMERIC;
BEGIN
    -- Ripristina la scorta del prodotto
    UPDATE prodotto
    SET scorta = scorta + OLD.numeroarticoli
```

```
    WHERE codprodotto = OLD.codprodotto;

    -- Recupera il cliente associato all'ordine
    SELECT codcliente INTO cod_cliente
    FROM ordine
    WHERE codordine = OLD.codordine;

    -- Calcola i punti da togliere (10% del valore speso)
    punti_da_togliere := ROUND(OLD.prezzo * OLD.numeroarticoli *
       0.10, 2);

    -- Aggiorna i punti della tessera del cliente
    UPDATE tessera
    SET numeropunti = GREATEST(numeropunti - punti_da_togliere,
       0)
    WHERE codcliente = cod_cliente;

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

Funzione AggiornaPrezzoOrdineDelete

```
CREATE OR REPLACE FUNCTION AggiornaPrezzoOrdineDelete()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE ordine
    SET prezzototale = GREATEST(prezzototale - (OLD.prezzo * OLD.
       numeroarticoli), 0)
    WHERE codordine = OLD.codordine;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

4.3 View e Funzioni

In questa sezione vengono presentate alcune **view** e **funzioni** SQL progettate per semplificare l'accesso ai dati, supportare l'analisi e la reportistica e facilitare la gestione del negozio.

Le **view** permettono di aggregare e filtrare informazioni di interesse senza scrivere query complesse ogni volta, mentre le **funzioni parametrizzate** consentono di ottenere dati personalizzati secondo criteri specifici, come categoria di prodotto o intervallo temporale.

4.3.1 Funzioni parametrizzate

```
CREATE OR REPLACE FUNCTION RicercaClientiPerCategoria(
    categoriaRicerca TIPOLOGIA DEFAULT NULL
)
RETURNS TABLE(
    codcliente INTEGER,
    nome VARCHAR(255),
    cognome VARCHAR(255),
    categoria TIPOLOGIA,
    punti_categoria NUMERIC,
    spesa_categoria NUMERIC,
    ordini_categoria BIGINT
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        c.codcliente,
        c.nome,
        c.cognome,
        p.categoria,
        ROUND(COALESCE(SUM(ao.prezzo * ao.numeroarticoli * 0.10),
            0), 2) AS punti_categoria,
        ROUND(COALESCE(SUM(ao.prezzo * ao.numeroarticoli), 0), 2)
            AS spesa_categoria,
        COUNT(DISTINCT o.codordine)::BIGINT AS ordini_categoria
    FROM cliente c
    JOIN ordine o ON c.codcliente = o.codcliente
    JOIN articoliordine ao ON o.codordine = ao.codordine
    JOIN prodotto p ON ao.codprodotto = p.codprodotto
    WHERE categoriaRicerca IS NULL OR p.categoria =
       categoriaRicerca
    GROUP BY c.codcliente, c.nome, c.cognome, p.categoria
    HAVING SUM(ao.prezzo * ao.numeroarticoli * 0.10) > 0
    ORDER BY c.codcliente, punti_categoria DESC;
END;
$$ LANGUAGE plpgsql;
```

Questa funzione restituisce, per ogni cliente e categoria (o per tutte le categorie se il parametro è NULL), la somma dei punti fedeltà accumulati, la spesa totale e il numero di ordini effettuati.

```
CREATE OR REPLACE FUNCTION DipendenteStatistiche(
    data_inizio DATE,
    data_fine DATE
)
RETURNS TABLE(
    nome TEXT,
    cognome TEXT,
    NumeroOrdini INTEGER,
```

```
    Introiti NUMERIC
) AS $$
    SELECT nome, cognome, NumeroOrdini, NULL AS Introiti
    FROM (
        SELECT D.nome, D.cognome, COUNT(O.codordine) AS
           NumeroOrdini
        FROM dipendente D
        JOIN ordine O ON O.coddipendente = D.coddipendente
        WHERE O.dataacquisto BETWEEN data_inizio AND data_fine
        GROUP BY D.nome, D.cognome
        ORDER BY NumeroOrdini DESC
        LIMIT 1
    ) AS TopVendite
    UNION ALL
    -- Dipendente con maggiori introiti
    SELECT nome, cognome, NULL AS NumeroOrdini, Introiti
    FROM (
        SELECT D.nome, D.cognome, SUM(O.prezzototale) AS Introiti
        FROM dipendente D
        JOIN ordine O ON O.coddipendente = D.coddipendente
        WHERE O.dataacquisto BETWEEN data_inizio AND data_fine
        GROUP BY D.nome, D.cognome
        ORDER BY Introiti DESC
        LIMIT 1
    ) AS TopIntroito;
$$ LANGUAGE sql;
```

Questa funzione individua il dipendente con il maggior numero di ordini e quello con il maggior introito in un dato intervallo temporale, restituendo due righe distinte.

4.3.2 View principali

```
CREATE OR REPLACE VIEW VwClientiTessere AS
SELECT c.codcliente, c.nome, c.cognome,
       t.numeropunti, t.stato, t.datascadenza
FROM cliente c
JOIN tessera t ON c.codcliente = t.codcliente;
```

Visualizza rapidamente il riepilogo dei punti fedeltà e lo stato della tessera per tutti i clienti.

```
CREATE OR REPLACE VIEW VwProdottiScortaBassa AS
SELECT codprodotto, nome, categoria, scorta
FROM prodotto
WHERE scorta < 20;
```

Evidenzia i prodotti con scorta inferiore a 20 unità, utile per la gestione degli ordini e del magazzino.

```
CREATE OR REPLACE VIEW VwOrdiniDettagliati AS
```

```sql
SELECT o.codordine, o.dataacquisto, o.prezzototale,
       c.nome AS nome_cliente, c.cognome AS cognome_cliente,
       d.nome AS nome_dipendente, d.cognome AS cognome_dipendente
          ,
       p.nome AS nome_prodotto, p.categoria, ao.numeroarticoli,
          ao.prezzo
FROM ordine o
JOIN cliente c ON o.codcliente = c.codcliente
JOIN dipendente d ON o.coddipendente = d.coddipendente
JOIN articoliordine ao ON o.codordine = ao.codordine
JOIN prodotto p ON ao.codprodotto = p.codprodotto;
```

Fornisce una vista completa degli ordini, includendo informazioni su cliente, dipendente e dettagli dei prodotti ordinati.

4.3.3 Esempi di utilizzo

- **Statistiche dipendenti su intervalli temporali diversi:**

```sql
SELECT * FROM DipendenteStatistiche(CURRENT_DATE - INTERVAL '
   3 months', CURRENT_DATE);
SELECT * FROM DipendenteStatistiche(CURRENT_DATE - INTERVAL '
   12 months', CURRENT_DATE);
```

- **Clienti e punti per tutte le categorie:**

```sql
SELECT * FROM RicercaClientiPerCategoria() ORDER BY
   codcliente, categoria;
SELECT * FROM RicercaClientiPerCategoria('FRUTTA')
WHERE punti_categoria BETWEEN 0 AND 100;
```

- **Consultazione delle view:**

```sql
SELECT * FROM VwClientiTessere ORDER BY codcliente;
SELECT * FROM VwProdottiScortaBassa ORDER BY scorta ASC;
SELECT * FROM VwOrdiniDettagliati ORDER BY codordine,
   nome_prodotto;
```

5 Popolamento e Dati di Esempio

Il popolamento del database è stato effettuato tramite lo script `Popolazione.sql`, che contiene tutti gli **INSERT** necessari per popolare le tabelle con dati di esempio realistici e coerenti con la struttura progettata. Questo file è fondamentale per testare il funzionamento delle query, dei trigger e delle funzioni implementate, simulando un ambiente operativo completo e credibile. **Cosa contiene Popolazione.sql?**

Lo script inserisce prodotti di tutte le tipologie (frutta, verdura, farinacei, latticini, uova, confezionati), ciascuno con attributi specifici per categoria. Vengono aggiunti clienti e relative tessere punti con saldo iniziale e stato, dipendenti con dati anagrafici completi, ordini associati a clienti e dipendenti, e articoli acquistati. Grazie ai trigger, vengono generati automaticamente i movimenti dei punti fedeltà. **Esempio di inserimento:**

```sql
-- Inserimento di un prodotto di tipo frutta
INSERT INTO prodotto (nome, descrizione, prezzo, luogoprovenienza
   , dataraccolta, categoria, scorta)
VALUES ('Mela', 'Mela Golden', 1.20, 'Trentino', '2025-07-10', '
   FRUTTA', 50);

-- Inserimento di un dipendente
INSERT INTO dipendente (nome, cognome, codicefiscale, indirizzo,
   telefono, email)
VALUES ('Giulia', 'Bianchi', 'BNCGLI90A41H501U', 'Via Milano 5',
   '3339876543', 'giulia.bianchi@email.it');

-- Inserimento di un cliente
INSERT INTO cliente (nome, cognome, codicefiscale, indirizzo,
   telefono, email)
VALUES ('Mario', 'Rossi', 'RSSMRA80A01H501U', 'Via Roma 1', '
   3331234567', 'mario.rossi@email.it');

-- Associazione tessera al cliente
INSERT INTO tessera (codcliente, numeropunti, dataemissione,
   datascadenza, stato)
VALUES (1, 10, '2025-07-01', '2027-07-01', 'ATTIVA');

-- Inserimento di un ordine associato a cliente e dipendente
INSERT INTO ordine (prezzototale, dataacquisto, codcliente,
   coddipendente)
VALUES (0, '2025-07-29', 1, 1);

-- Inserimento di articoli nell'ordine (acquisto di 3 mele)
INSERT INTO articoliordine (codordine, codprodotto, prezzo,
   numeroarticoli)
VALUES (1, 1, 1.20, 3);
```

Questo esempio mostra un ciclo completo: creazione di prodotto, dipendente, cliente, tessera, ordine e articoli acquistati. I trigger e le funzioni implementate aggiorneranno

automaticamente la scorta del prodotto, i punti della tessera e il totale dell'ordine. **Come utilizzare lo script:**
Eseguire `Popolazione.sql` dopo la creazione delle tabelle per ottenere un database pronto all'uso, completo di dati di esempio per testare tutte le funzionalità implementate. I dati inseriti permettono di verificare le query di ricerca, le statistiche e la gestione dei punti fedeltà, simulando scenari reali di vendita e gestione clienti.